

UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Dipartimento di Scienze Fisiche,
Informatiche e Matematiche

HPC Labs – The Fine art of Profiling

High-Performance Computing [262-022]

Corso di Laurea in INFORMATICA
(D.M.270/04) [16-262]
Anno accademico 2026/2027

Prof. Alessandro Capotondi
a.capotondi@unimore.it

Copyright note

È vietata la copia e la riproduzione dei contenuti e immagini in qualsiasi forma.

È inoltre vietata la redistribuzione e la pubblicazione dei contenuti e immagini non autorizzata espressamente dall'autore o dall'Università di Modena e Reggio Emilia.



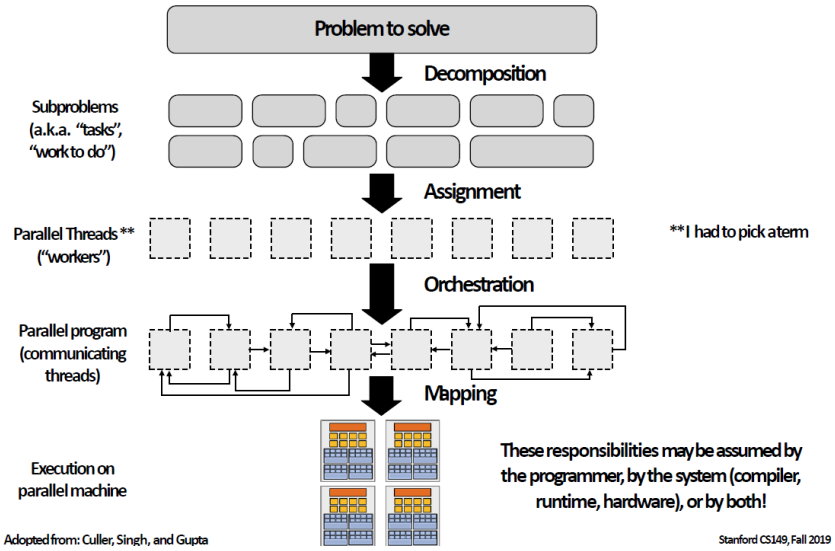
If you know the enemy and know yourself you need not fear the results of a hundred battles.

- Sun Tzu

What does it mean know yourself and know your enemy?

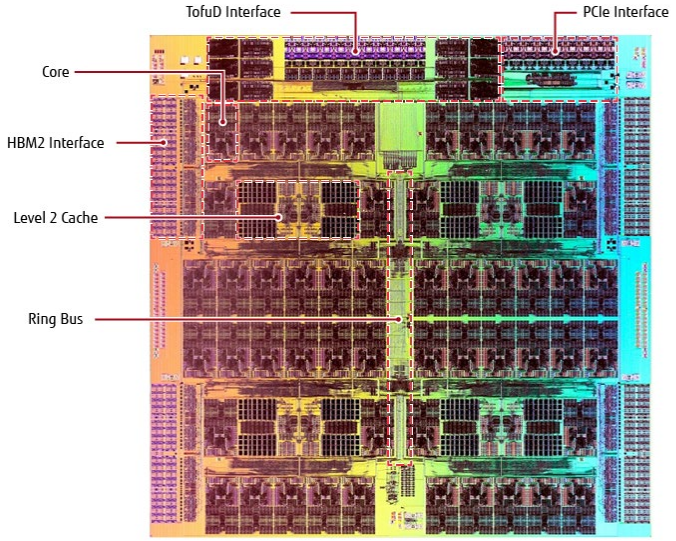
Create a parallel program means

- **Identify** work that can be performed in parallel **(Analyze)**
  - Identify HOTSPOTS (Amdahl's law)
- **Partition** the work and **Synchronize (Code)**
  - Performance limited in bandwidth or by Computation?
- **Assessment** of the performance **(Evaluate)**
  - Identify metrics (FLOPS, FPS, latency)



Yourself is your system, your HW

- **What is the maximum performance of your systems?**
  - Throughput: FLOPS
  - Memory Bandwidth: GB/s
  - Energy Efficiency: FLOPS/W
- **What are the features?**
  - How many threads/cores has it?
  - Does have Vector Unit?
  - Does have HW specific Accelerators? (NN)



Your enemy is the application!

- **How many operations I must do?**
  - How it scales?
- **How much data I must access?**
  - Which data layout?
  - 2D, 3D, sparse?
  - Double, float, int
- **Which function costs more?**
  - Where are my hot spots?
  - Where I spent more time?

How to know about yourself?

Roofline model*

https://people.eecs.berkeley.edu/~kubitron/cs252/handouts/papers/RooflineVyNoYellow.pdf LINK

- Use microbenchmarks to compute peak performance of a machine as a function of arithmetic intensity of application
- Then compare application's performance to known peak values

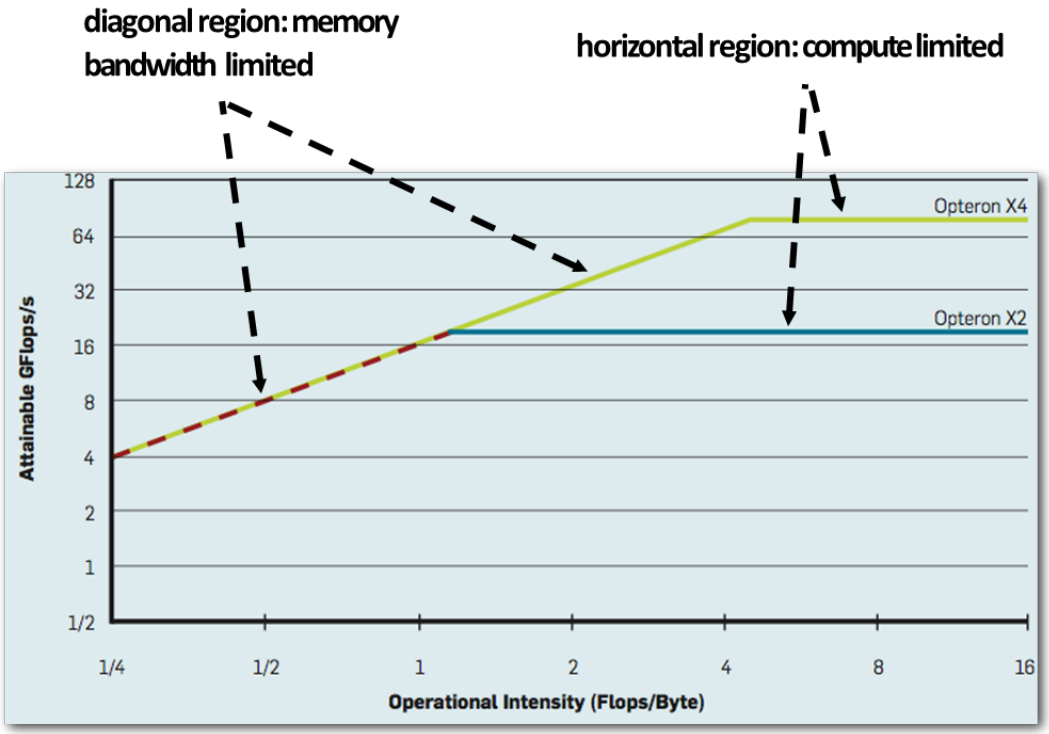


Figure credit: Williams et al. 2009

How to know about yourself?

Roofline model: optimization regions

- Use various levels of optimization in benchmarks
  (e.g., best performance with and without using SIMD instructions)

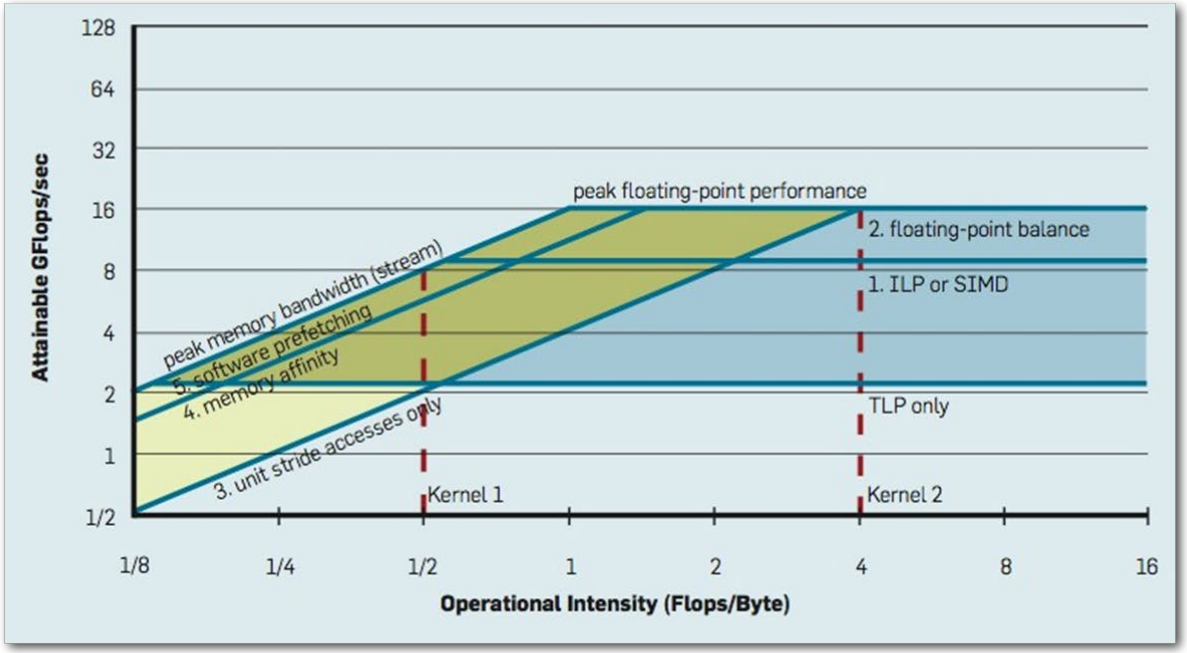


Figure credit: Williams et al. 2009

Example of metrics for NN systems

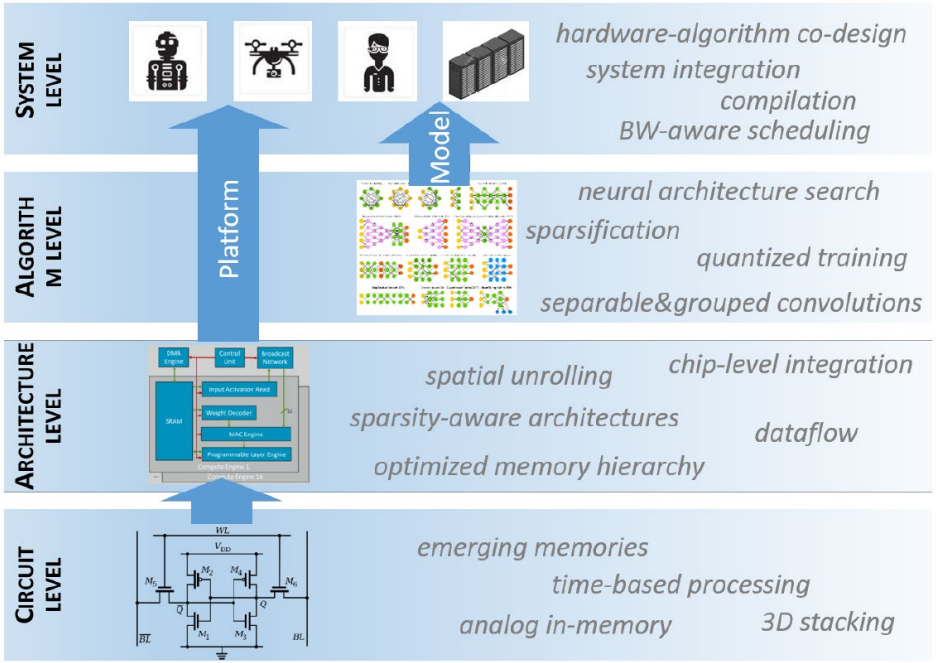
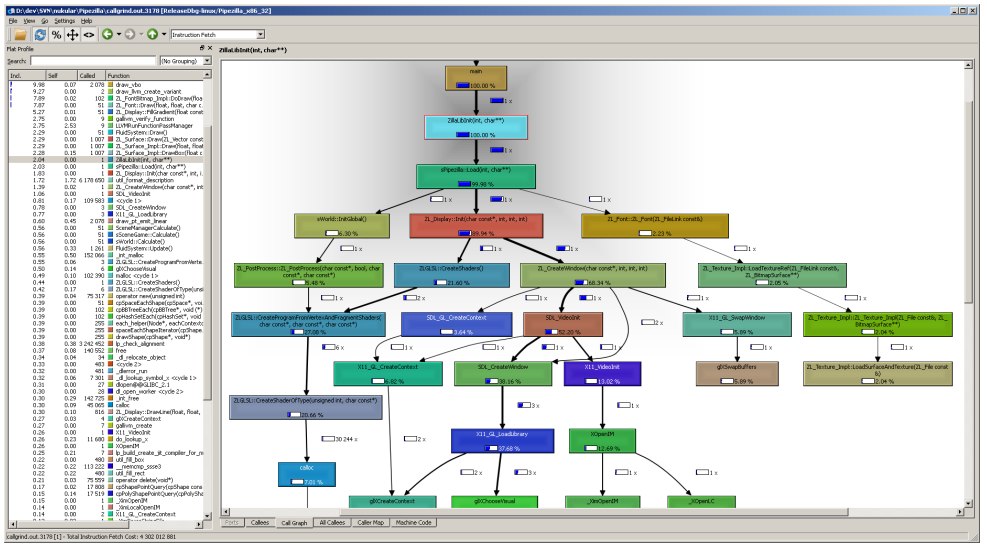


TABLE I
MAIN CONSTRAINTS AND MEASURABLE METRICS ACROSS BENCHMARKING ABSTRACTION LEVELS.
(1 TOPS = (10^{12} OR 1 TERA)-OPERATIONS-PER-SECOND;
1 MAC/SEC = 1 MULTIPLY-ACCUMULATE-PER-SECOND)

System	Architecture	Circuit
accuracy inf/sec energy/inf cost	kernel accuracy TOPS (utilization) TOPS/W (mean & peak) TOPS/mm^2 (mean & peak)	MAC accuracy MAC/sec pJ/MAC (peak) MAC/sec/mm^2 (peak)
	@ system-induced precision, sparsity, model topology, with system-level implications stated clearly.	

How you know about your enemy?

- Profile, profile, profile
- Evaluate performance
- Find the performance bottlenecks
  - inefficient programming
  - memory, I/O bottlenecks
  - Vectorization
  - parallel scaling
- Other?



How to know about your enemy?

- **Profiling is the collection of information about a program execution for selected inputs and a given platform.** It can measure or estimate:
  - Execution time and CPU time
  - Memory usage
  - Executed instructions and hardware events
  - Function call counts and call relationships

Remember:

Profiling results are input-dependent and platform-dependent.

They do not, by themselves, prove asymptotic complexity or worst-case execution time.

Dynamic or static?

Why dynamic?

- Some applications duration depend on the input data statistics
  - (static analysis can be not effective to find hotspots)
- Instructions can have different execution time
  - (you can’t just count the number of instructions)
- Complex memory hierarchies are not easy to predict
  - (shared multi level cache performances can be affected by the execution of other tasks)
- Others?

Techniques for collecting profiling data

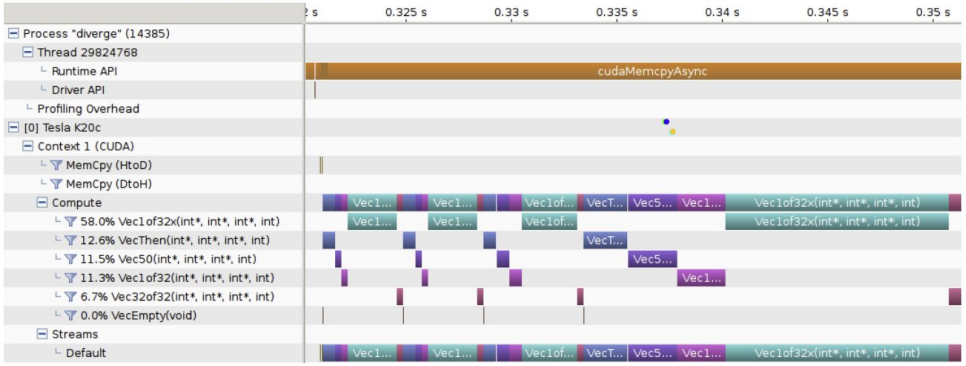
- **Performance Counters**
  - clock cycles, timers, cache hits, etc.
  - Need deep understanding of the HW
- **Code Instrumentation**
  - Automatic and/or manual
  - Heisenberg effect: counters/instrumentation can perturb execution time
- **Hardware Interrupts**
  - Time-based profiling (interrupt every t seconds)
- **Operative System hooks**
  - Hook to specific OS events (scheduling points, context switch)
- **ISA simulators**
  - Use a simulator to have full introspection of the code execution
  - Very slow

Profiler outputs

- **Statistical summary** of the events

% time	cumulative seconds	self seconds	calls	self ms/call	total ms/call	name
70.45	5.14	5.14	26074562	0.00	0.00	intersect
26.01	7.03	1.90	4000000	0.00	0.00	shade
3.72	7.30	0.27	100	2.71	73.03	calc_tile

- **Trace** of the recorded events



- **Live** monitoring



What will see today?

- **perf** tools (based on counters)
  - https://perf.wiki.kernel.org/index.php/Tutorial
  - Needs prior machine configuration

```
perf ….
```

- **GNU gprof** (compiler instrumentation)
  - Widely available with UNIX/Linux distributions

```
gcc -O2 -pg foo.c -o foo
./foo
gprof foo
```

Valgrind and its tools: http://valgrind.org/info/tools.html

```
valgrind --tool=callgrind ./foo
kcachegrind to visualize
```

Perf

- perf: Linux profiling with performance counters
- Perf is a profiler tool for Linux 2.6+ based systems that abstracts away CPU hardware differences in Linux performance measurements and presents a simple command line interface.
- Perf is based on the **perf_events** interface exported by recent versions of the Linux kernel.
- Needs machine configuration
- Installation

```
sudo apt install linux-tools-generic
```

Perf - Let’s try

- Login to the board
- Load the perf environment

```
module load perf/1.0
```

Perf Commands

```
perf

 usage: perf [--version] [--help] COMMAND [ARGS]

 The most commonly used perf commands are:
  annotate        Read perf.data (created by perf record) and display annotated code
  archive         Create archive with object files with build-ids found in perf.data file
  bench           General framework for benchmark suites
  buildid-cache   Manage <tt>build-id</tt> cache.
  buildid-list    List the buildids in a perf.data file
  diff            Read two perf.data files and display the differential profile
  inject          Filter to augment the events stream with additional information
  kmem            Tool to trace/measure kernel memory(slab) properties
  kvm             Tool to trace/measure kvm guest os
  list            List all symbolic event types
  lock            Analyze lock events
  probe           Define new dynamic tracepoints
  record          Run a command and record its profile into perf.data
  report          Read perf.data (created by perf record) and display the profile
  sched           Tool to trace/measure scheduler properties (latencies)
  script          Read perf.data (created by perf record) and display trace output
  stat            Run a command and gather performance counter statistics
  test            Runs sanity tests.
  timechart       Tool to visualize total system behavior during a workload
  top             System profiling tool.

 See 'perf help COMMAND' for more information on a specific command.
```

Perf Commands help

- Certain commands require special support in the kernel and may not be available.
- To obtain the list of options for each command, simply type the command name followed by -h:

```
perf stat -h

 usage: perf stat [<options>] [<command>]

    -e, --event <event>   event selector. use 'perf list' to list available events
    -i, --no-inherit      child tasks do not inherit counters
    -p, --pid <n>         stat events on existing process id
    -t, --tid <n>         stat events on existing thread id
    -a, --all-cpus        system-wide collection from all CPUs
    -c, --scale           scale/normalize counters
    -v, --verbose         be more verbose (show counter open errors, etc)
    -r, --repeat <n>      repeat command and print average + stddev (max: 100)
    -n, --null            null run - dont start any counters
    -B, --big-num         print large numbers with thousands' separators
```

Perf Events

- The perf tool supports a list of measurable events:
  - **Software Events (mainly from Linux Kernel)**
    - Kernel Counters
      - # context switches
      - Faults
      - Etc.
  - **Hardware Events (Performance Monitor Units – PMU)**
    - Number of cycles,
    - Instructions retired
    - L1 cache misses
- To **obtain** the list of supported events:

```
perf list

List of pre-defined events (to be used in -e):

 cpu-cycles OR cycles                       [Hardware event]
 instructions                               [Hardware event]
 cache-references                           [Hardware event]
 cache-misses                               [Hardware event]
 branch-instructions OR branches            [Hardware event]
 branch-misses                              [Hardware event]
 bus-cycles                                 [Hardware event]
```

Performance Monitor Unit Events (2)

- PMU hardware events are CPU specific and documented by the CPU vendor.

- The perf tool, if linked against the **libpfm4** library, provides some short description of the events. For a listing of PMU hardware events for Intel, AMD processors, and ARM see:
  - Intel PMU event tables: LINK
  - AMD PMU event table: LINK
  - ARM PMU: LINK

Perf – Which events in the Nvidia Tegra?

perf list

```
List of pre-defined events (to be used in -e):

  branch-misses                                      [Hardware event]
  bus-cycles                                         [Hardware event]
  cache-misses                                       [Hardware event]
  cache-references                                   [Hardware event]
  cpu-cycles OR cycles                               [Hardware event]
  instructions                                       [Hardware event]

  alignment-faults                                   [Software event]
  bpf-output                                         [Software event]
  context-switches OR cs                             [Software event]
  cpu-clock                                          [Software event]
  cpu-migrations OR migrations                       [Software event]
  dummy                                              [Software event]
  emulation-faults                                   [Software event]
  major-faults                                       [Software event]
  minor-faults                                       [Software event]
  page-faults OR faults                              [Software event]
  task-clock                                         [Software event]

  L1-dcache-load-misses                              [Hardware cache event]
  L1-dcache-loads                                    [Hardware cache event]
  L1-dcache-store-misses                             [Hardware cache event]
  L1-dcache-stores                                   [Hardware cache event]
  L1-icache-load-misses                              [Hardware cache event]
  L1-icache-loads                                    [Hardware cache event]
  branch-load-misses                                 [Hardware cache event]
  branch-loads                                       [Hardware cache event]
  dTLB-load-misses                                   [Hardware cache event]
  iTLB-load-misses                                   [Hardware cache event]

  armv8_pmuv3/br_mis_pred/                           [Kernel PMU event]
  armv8_pmuv3/br_pred/                               [Kernel PMU event]
  armv8_pmuv3/bus_access/                            [Kernel PMU event]
  armv8_pmuv3/bus_cycles/                            [Kernel PMU event]
  armv8_pmuv3/cid_write_retired/                     [Kernel PMU event]
  armv8_pmuv3/cpu_cycles/                            [Kernel PMU event]
  armv8_pmuv3/exc_return/                            [Kernel PMU event]
  armv8_pmuv3/exc_taken/                             [Kernel PMU event]
  armv8_pmuv3/inst_retired/                          [Kernel PMU event]
  armv8_pmuv3/inst_spec/                             [Kernel PMU event]
  armv8_pmuv3/l1d_cache/                             [Kernel PMU event]
  armv8_pmuv3/l1d_cache_refill/                      [Kernel PMU event]
  armv8_pmuv3/l1d_cache_wb/                          [Kernel PMU event]
  armv8_pmuv3/l1d_tlb_refill/                        [Kernel PMU event]
  armv8_pmuv3/l1i_cache/                             [Kernel PMU event]
  armv8_pmuv3/l1i_cache_refill/                      [Kernel PMU event]
  armv8_pmuv3/l1i_tlb_refill/                        [Kernel PMU event]
  armv8_pmuv3/l2d_cache/                             [Kernel PMU event]
  armv8_pmuv3/l2d_cache_refill/                      [Kernel PMU event]
  armv8_pmuv3/l2d_cache_wb/                          [Kernel PMU event]
  armv8_pmuv3/mem_access/                            [Kernel PMU event]
  armv8_pmuv3/memory_error/                          [Kernel PMU event]
  armv8_pmuv3/sw_incr/                               [Kernel PMU event]
  armv8_pmuv3/ttbr_write_retired/                    [Kernel PMU event]

  rNNN                                               [Raw hardware event descriptor]
```

Perf – Let's try (2)

- **Command**

perf stat <COMMAND>

- **Profile using stats**

perf stat dd if=/dev/zero of=/dev/null count=1000000

```
alessandro.capotondi@nano:~$ perf stat dd if=/dev/zero of=/dev/null count=1000000
1000000+0 records in
1000000+0 records out
512000000 bytes (512 MB, 488 MiB) copied, 1.83649 s, 279 MB/s

 Performance counter stats for 'dd if=/dev/zero of=/dev/null count=1000000':

       1840.153912      task-clock (msec)         #    0.998 CPUs utilized
                 3      context-switches          #    0.002 K/sec
                 0      cpu-migrations            #    0.000 K/sec
                65      page-faults               #    0.035 K/sec
     2,712,980,223      cycles                    #    1.474 GHz
     1,708,933,911      instructions              #    0.63  insn per cycle
   <not supported>      branches
         3,029,908      branch-misses

       1.843373880 seconds time elapsed
```

Perf Modifiers

Modifiers

Events can optionally have a modifier by appending a colon and one or more modifiers. Modifiers allow the user to restrict when events are counted.

To measure a PMU event and pass modifiers:

```
perf stat -e instructions:u dd if=/dev/zero of=/dev/null count=100000
```

In this example, we are measuring the number of instructions at the user level. Note that for actual events, the modifiers depends on the underlying PMU model. All modifiers can be combined at will. Here is a simple table to summarize the most common modifiers for Intel and AMD x86 processors.

Modifiers	Description	Example
u	monitor at priv level 3, 2, 1 (user)	event:u
k	monitor at priv level 0 (kernel)	event:k
h	monitor hypervisor events on a virtualization environment	event:h
H	monitor host machine on a virtualization environment	event:H
G	monitor guest machine on a virtualization environment	event:G

All modifiers above are considered as a boolean (flag).

Perf Repeated Measures

Repeated measurement

It is possible to use `perf stat` to run the same test workload multiple times and get for each count, the standard deviation from the mean.

```
perf stat -r 5 sleep 1

 Performance counter stats for 'sleep 1' (5 runs):

    <not counted> cache-misses
           20,676 cache-references          #     13.046 M/sec    ( +-   0.658% )
            6,229 branch-misses             #      0.000 %        ( +-  40.825% )
    <not counted> branches
    <not counted> instructions
    <not counted> cycles
              144 page-faults               #      0.091 M/sec    ( +-   0.139% )
                0 CPU-migrations            #      0.000 M/sec    ( +-    -nan% )
                1 context-switches          #      0.001 M/sec    ( +-   0.000% )
        1.584872 task-clock-msecs           #      0.002 CPUs     ( +-  12.480% )

       1.002251432  seconds time elapsed    ( +-   0.025% )
```

Here, `sleep` is run 5 times and the mean count for each event, along with ratio of std-dev/mean is printed.

Measurement protocol

For a fair comparison:

- Use the same input size, data type, compiler options, and number of threads.
- Verify that both implementations produce equivalent results before comparing performance.
- Define the measured region: whole program or kernel only.
- Run one or more warm-up executions before collecting data.
- Repeat measurements and report the individual runs or their variability.
- Alternate the execution order of the implementations.
- Record the board, software versions, CPU frequency or power mode, and temperature.
- Only compare measurements collected over the same execution region.

Perf – Let's try (3)

- **Command**

  dd if=/dev/zero of=/dev/null count=1000000

- **Profile the average stats in 5 executions**

```
perf stat -r 5 <COMMAND>
```

- **Profile instructions and cycles in user-space w.r.t to all**

```
perf -e cycles:u,instructions:u 5 <COMMAND>
```

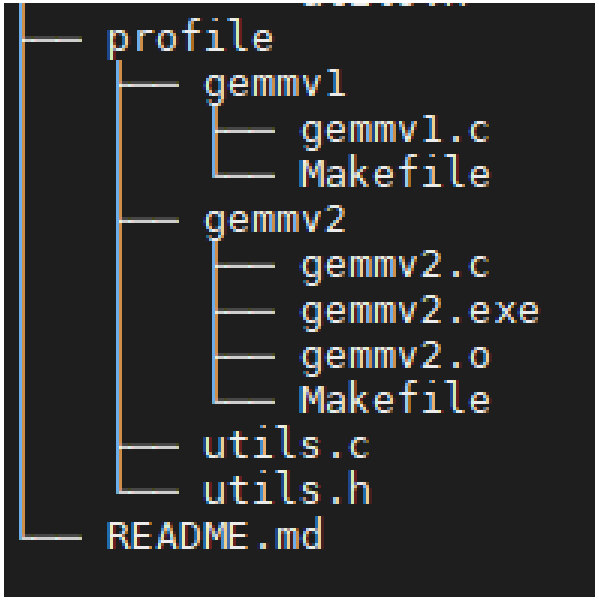
Download, Compile and Run

1. Connect to your assigned Nano board: (See HPC-LAB-HOWTO)

```
ssh –X –p PORT <USER>@<HOST>
```

2. Download Sources

```
git clone https://git.hipert.unimore.it/hpc_class/hpc-lab-public-sources.git
```



3. Compile and Run

```
cd hpc-lab-public-sources/profile/gemmv1
make clean all
```

```
cd hpc-lab-public-sources/profile/gemmv2
make clean all
```

Exercise

- Profile the two applications
  - Cycles
  - IPC
  - Instructions
  - Memory Accesses

- Which implementation is faster under the tested conditions?
- Which measurements support your explanation?

Perf – many more things…

- You can annotate code
- You can monitor running process
- You can monitor the Linux kernel itself
- See documentation: https://perf.wiki.kernel.org/index.php/Main_Page

Perf – Pros and Cons

Pros

- Very fine grain profiling information
- A lot of information

Cons

- Events may vary on different architectures
- Too much information is not always good!

GNU Gprof

- Part of the GNU binutils tools
- Provides data on number of calls and time spent in each function in program

```
Flat profile:


Each sample counts as 0.01 seconds.
%    cumulative self          self   total
time seconds    seconds calls s/call s/call name
33.86 15.52     15.52    1    15.52  15.52  func2
33.82 31.02     15.50    1    15.50  15.50  new_func1
33.29 46.27     15.26    1    15.26  30.75  func1
0.07  46.30     0.03                       main
```

```
Call graph (explanation follows)


granularity: each sample hit covers 2 byte(s) for 0.02% of 46.30 seconds


index % time self children called name


[1]   100.0  0.03  46.27          main [1]
             15.26 15.50    1/1      func1 [2]
             15.52 0.00     1/1      func2 [3]
-----------------------------------------------
             15.26 15.50    1/1      main [1]
[2]   66.4   15.26 15.50    1     func1 [2]
             15.50 0.00     1/1      new_func1 [4]
-----------------------------------------------
             15.52 0.00     1/1      main [1]
[3]   33.5   15.52 0.00     1     func2 [3]
-----------------------------------------------
             15.50 0.00     1/1      func1 [2]
[4] 33.5     15.50 0.00     1     new_func1 [4]
-----------------------------------------------
```

How to use GNU gprof

- Using the gprof tool is not at all complex. You just need to do the following on a high-level:

1. **Have profiling enabled** while compiling the code (-pg)
2. **Execute the program** code to produce the profiling data (gmon.out)
3. **Run the gprof tool** on the profiling data file (generated in the step above).

GNU Gprof – How it works

- **How gprof estimates time**

- gprof combines two sources of information:
  - Instrumentation records function calls and caller/callee relationships.
  - Periodic sampling estimates where CPU time is spent.

Example:
- If sampling occurs at 100 Hz, 1000 samples correspond to approximately 10 seconds of sampled CPU time.
- If function B receives 500 samples, its estimated self time is approximately 5 seconds.
- The 100 Hz value is only an example. The sampling frequency and accuracy depend on the system.

Important limitation
The children time attributed to function A is an estimate.

gprof assumes that the average cost of a function B does not depend on the caller.
This can be inaccurate when different callers invoke B with different inputs or different workloads.

Therefore, use call-graph children times as guidance for investigation, not as exact per-call measurements.

Exercise

```
cd hpc-lab-public-sources/profile/gprof
```

- Examine the profile.c and compile.sh code
- Compile it

```
./compile.sh
```

- Execute the profile application and run gprof

```
./profile
gprof ./profile
```

Output (Flat Profile)

```
$ ./profile

Inside the function main()

Inside the function test()

Inside the function another_test()

Inside the function yet_other_test()

Inside the function some_other_test()
```

Resolution

```
Flat profile:

Each sample counts as 0.01 seconds.
  %   cumulative   self              self     total
 time   seconds   seconds    calls  ms/call  ms/call  name
 94.58      0.32     0.32        1   321.58   321.58  yet_another_test
  5.91      0.34     0.02        1    20.10   341.67  test
  0.00      0.34     0.00        1     0.00   321.58  another_test
  0.00      0.34     0.00        1     0.00     0.00  some_other_test
```

The statistics displayed above is known as flat profile. Here is the explanation (taken from the file output) of what each column means :

- **% time** - The percentage of the total running time of the program used by this function.
- **Cumulative seconds** - A running sum of the number of seconds accounted for by this function and those listed above it.
- **Self seconds** - The number of seconds accounted for by this function alone. This is the major sort for this listing.
- **Calls** - The number of times this function was invoked, if this function is profiled, else blank.
- **Self ms/call** - The average number of milliseconds spent in this function per call, if this function is profiled, else blank.
- **Total ms/call** - The average number of milliseconds spent in this function and its descendent per call, if this function is profiled, else blank.
- **Name** - The name of the function. This is the minor sort for this listing. The index shows the location of the function in the gprof listing. If the index is in parenthesis, it shows where it would appear in the gprof listing if it were to be printed.

Output (Graph Profile)

```
granularity: each sample hit covers 2 byte(s) for 2.93% of 0.34 seconds

index % time    self  children    called     name
                0.02    0.32       1/1          main [2]
[1]    100.0    0.02    0.32       1          test [1]
                0.00    0.32       1/1          another_test [3]
-----------------------------------------------
                                               <spontaneous>
[2]    100.0    0.00    0.34                 main [2]
                0.02    0.32       1/1          test [1]
                0.00    0.00       1/1          some_other_test [5]
-----------------------------------------------
                0.00    0.32       1/1          test [1]
[3]     94.1    0.00    0.32       1          another_test [3]
                0.32    0.00       1/1          yet_another_test [4]
-----------------------------------------------
                0.32    0.00       1/1          another_test [3]
[4]     94.1    0.32    0.00       1          yet_another_test [4]
-----------------------------------------------
                0.00    0.00       1/1          main [2]
[5]      0.0    0.00    0.00       1          some_other_test [5]
-----------------------------------------------
```

This table describes the call tree of the program and was sorted by the total amount of time spent in each function and its children.

- **Index** - A unique number given to each element of the table. Index numbers are sorted numerically. The index number is printed next to every function name, so it is easier to look up where the function is in the table.
- **% time** - This is the percentage of the `total' time that was spent in this function and its children. Note that due to different viewpoints, functions excluded by options, etc., these numbers will NOT add up to 100%.
- **Self** - This is the total amount of time spent in this function
- **Children** - This is the total amount of time propagated into this function by its children.
- **Called** - This is the number of times the function was called.

GNU Gprof – Pros and Cons

Pros

- Very easy to use
- Good for find hotspots

Cons

- No information about why we spent so much time in some function
- Call tree representation is not intuitive (at all)

More info:

- Manual: LINK
- Paper: LINK

Valgrind



- The Valgrind distribution includes the following debugging and profiling tools:
- Memcheck (memcheck)
- Cachegrind (cache profiling)
- Callgrind (function call profiling)
- Many others
  - Massif (heap)
  - Helgrind (thread debugger)
  - DRD (multi-thread error detector)
  - …

Memcheck

- Memcheck detects memory-management problems, and is aimed primarily at C and C++ programs
  - Accesses memory it shouldn't (areas not yet allocated, areas that have been freed, areas past the end of heap blocks, inaccessible areas of the stack)
  - Uses uninitialised values in dangerous ways
  - Leaks memory
  - Does bad frees of heap blocks (double frees, mismatched frees)
  - Passes overlapping source and destination memory blocks to memcpy() and related functions

Cachegrind and Callgrind

- **Cachegrind** is a tracing profiler. It reports executed instruction counts and can optionally simulate cache and branch behavior.

  When cache simulation is enabled, Cachegrind models:

  I1: first-level instruction cache

  D1: first-level data cache

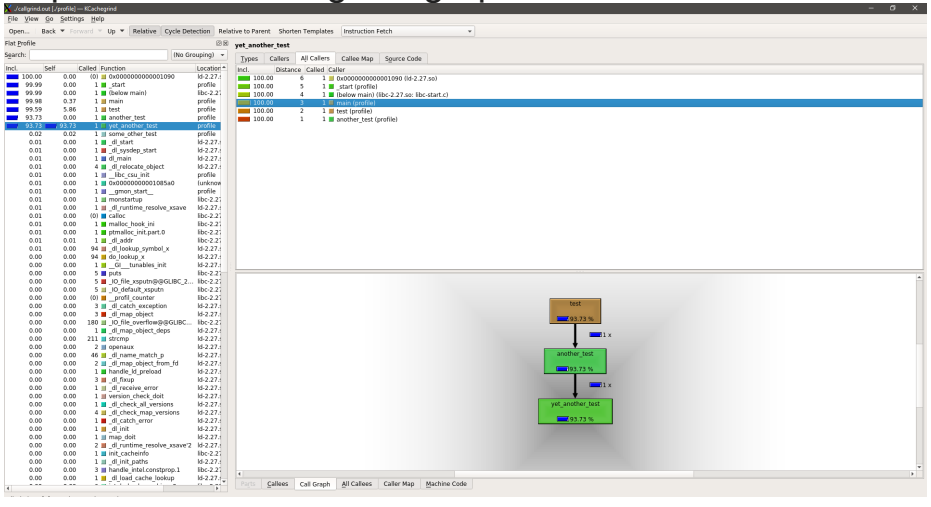
  LL: last-level cache

- The simulated miss counts depend on the selected cache model. They are not direct measurements of cache misses on the target processor.

- **Callgrind** records caller/callee relationships and instruction costs. Cache and branch simulation are optional features.

-

- **KCachegrind** can be used to explore the resulting call graph and event costs.

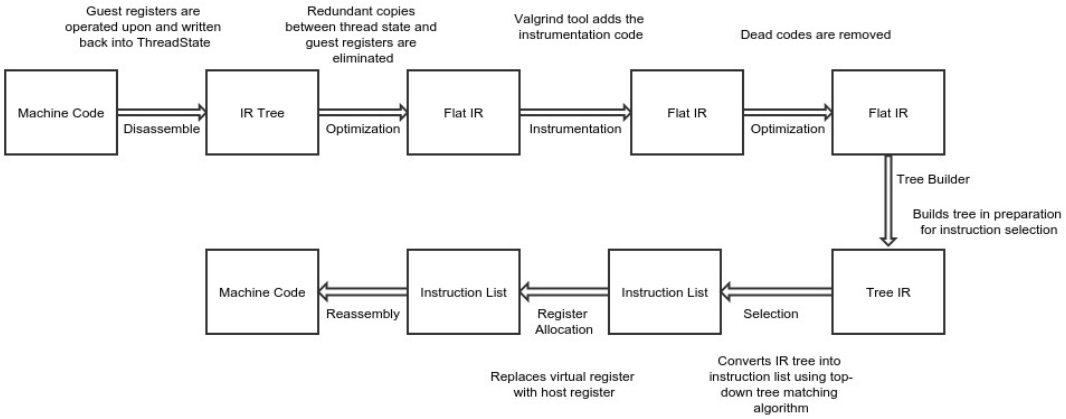


How it works

- Valgrind runs your application in a "sandbox." While running in this sandbox, it is able to insert its own instructions to do advanced debugging and profiling.

- Valgrind is a Dynamic Binary Analysis (DPA) tool that uses **Dynamic Binary Instrumentation (DPI)** framework to check memory allocation, to detect deadlocks and to profile the applications. DPI framework has its own low level memory manager, scheduler, thread handler and signal handler.

- **Valgrind Core** disassembles the application code and passes the code fragment to tool plugin for instrumentation. The tool plugin adds the analysis code and assembles it back.



Installation & Usage

- Installation

```
sudo apt install valgrind kcachegrind
```

- Usage (callgrind)

```
valgrind --tool=cachegrind  <APPLICATION> <args>
kcachegrind
```

- Usage (callgrind)

```
valgrind --tool=callgrind  <APPLICATION> <args>
kcachegrind
```

Exercise

- Profile with valgrind (--tool=cachegrind)
  - gemmv1
  - gemmv2
- Profile with valgrind (--tool=callgrind)
  - gprof application

Gemm cachegrind

Gemmv1

```
alessandro.capotondi@nano:~/hpc-lab-public-sources/profile/gemmv1$ valgrind --
==23508== Cachegrind, a cache and branch-prediction profiler
==23508== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al
==23508== Using Valgrind-3.13.0 and LibVEX; rerun with -h for copyright info
==23508== Command: ./gemmv1.exe
==23508==
--23508-- Warning: Cannot auto-detect cache config, using defaults.
--23508--          Run with -v to see.
gemm on host :   248.261 sec       8.7 MFLOPS
==23508==
==23508== Process terminating with default action of signal 27 (SIGPROF)
==23508==    at 0x49E265C: __open_nocancel (open64.c:69)
==23508==    by 0x49F2057: write_gmon (gmon.c:370)
==23508==    by 0x49F2697: _mcleanup (gmon.c:444)
==23508==    by 0x49554D7: __cxa_finalize (cxa_finalize.c:83)
==23508==    by 0x108E8F: ??? (in /nfs/home/alessandro.capotondi/hpc-lab-publi
==23508==
==23508== I   refs:      7,527,150,213
==23508== I1  misses:            1,249
==23508== LLi misses:            1,087
==23508== I1  miss rate:          0.00%
==23508== LLi miss rate:          0.00%
==23508==
==23508== D   refs:      2,149,728,958  (2,148,652,976 rd   + 1,075,982 wr)
==23508== D1  misses:    1,140,968,900  (1,140,968,135 rd   +       765 wr)
==23508== LLd misses:    1,140,967,946  (1,140,967,263 rd   +       683 wr)
==23508== D1  miss rate:          53.1% (         53.1%     +       0.1%  )
==23508== LLd miss rate:          53.1% (         53.1%     +       0.1%  )
==23508==
==23508== LL refs:       1,140,970,149  (1,140,969,384 rd   +       765 wr)
==23508== LL misses:     1,140,969,033  (1,140,968,350 rd   +       683 wr)
==23508== LL miss rate:           11.8% (         11.8%     +       0.1%  )
Profiling timer expired
```

Gemmv2

```
alessandro.capotondi@nano:~/hpc-lab-public-sources/profile/gemmv1$ valgrind --tool=
valgrind: ../gemmv2/: is a directory
alessandro.capotondi@nano:~/hpc-lab-public-sources/profile/gemmv1$ valgrind --tool=
==23774== Cachegrind, a cache and branch-prediction profiler
==23774== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
==23774== Using Valgrind-3.13.0 and LibVEX; rerun with -h for copyright info
==23774== Command: ../gemmv2/gemmv2.exe
==23774==
--23774-- Warning: Cannot auto-detect cache config, using defaults.
--23774--          Run with -v to see.
gemm_opt on host :   116.952 sec      18.4 MFLOPS
==23774==
==23774== Process terminating with default action of signal 27 (SIGPROF)
==23774==    at 0x49E265C: __open_nocancel (open64.c:69)
==23774==    by 0x49F2057: write_gmon (gmon.c:370)
==23774==    by 0x49F2697: _mcleanup (gmon.c:444)
==23774==    by 0x49554D7: __cxa_finalize (cxa_finalize.c:83)
==23774==    by 0x108E8F: ??? (in /nfs/home/alessandro.capotondi/hpc-lab-public-sou
==23774==
==23774== I   refs:      7,704,005,230
==23774== I1  misses:            1,259
==23774== LLi misses:            1,094
==23774== I1  miss rate:          0.00%
==23774== LLi miss rate:          0.00%
==23774==
==23774== D   refs:      3,271,669,387  (2,181,128,013 rd   + 1,090,541,374 wr)
==23774== D1  misses:       15,321,616  (   15,168,252 rd   +       153,364 wr)
==23774== LLd misses:        3,415,330  (    3,411,188 rd   +         4,142 wr)
==23774== D1  miss rate:           0.5% (          0.7%     +           0.0%  )
==23774== LLd miss rate:           0.1% (          0.2%     +           0.0%  )
==23774==
==23774== LL refs:          15,322,875  (   15,169,511 rd   +       153,364 wr)
==23774== LL misses:         3,416,424  (    3,412,282 rd   +         4,142 wr)
==23774== LL miss rate:            0.0% (          0.0%     +           0.0%  )
Profiling timer expired
```

Valgrind – Pros and Cons

Pros

- Very wide range of functionalities
- Intuitive and easy to use
- Useful also for debugging

Cons

- Execution is slower than normal code
- You can expect slightly different behavior compare regular code

Other profiling tools

- **Community**
  - Oprofile
    - https://oprofile.sourceforge.io/news/
  - HPC Toolkit
    - http://hpctoolkit.org/
  - TAU
    - http://www.cs.uoregon.edu/research/tau/home.php
- **Vendor Specific**
  - Intel Vtune
    - https://www.intel.com/content/www/us/en/developer/tools/oneapi/vtune-profiler.html
  - Nvidia Nsight
    - https://developer.nvidia.com/nsight-visual-studio-edition
  - AMD uProf
    - https://developer.amd.com/amd-uprof/

Conclusions

- Profiling is an important process of software development
- Exists different tools for profiling, for different level of abstraction
- Continuous assessment of the application performances is the key for deliver efficient high-performance code



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Dipartimento di Scienze Fisiche,
Informatiche e Matematiche

HPC Labs - Fine art of Profiling

High-Performance Computing [262-022]

Corso di Laurea in INFORMATICA
(D.M.270/04) [16-262]
Anno accademico 2025/2026

Prof. Alessandro Capotondi
a.capotondi@unimore.it